Introduction to Programming

COL 1000 Lecture Notes

Department of Computer Science & Engineering
IIT Delhi

Contents

Chapter 1

Introduction and Python Basics

1.1 What Can a Computer Do?

A computer is fundamentally a machine that can perform two primary operations:

1. **Perform Calculations**: Computers can execute elementary mathematical operations very quickly, including addition, subtraction, multiplication, and division. These operations can be repeated thousands of millions of times per second.
2. **Remember Results**: Computers have the ability to store intermediate results of computations in memory for later use. They can remember enormous amounts of data, far beyond what a human could memorize.

These simple operations can be combined and sequenced to solve complex computational problems. The key insight is that computers can do many fast calculations and have substantial storage capacity, but they require precise, well-defined instructions to operate.

1.2 Capabilities Built Upon Elementary Operations

Beyond basic arithmetic, we can build more complex capabilities through composition:

- **Built-in Elementary Operations**

  These are operations defined by the programming language itself (e.g., arithmetic operations $+$, $-$, $*$, $/$).

- **User-Defined Complex Calculations**

  These are sophisticated algorithms that we create by combining elementary operations. For example, computing the average of a list involves adding all elements (using '+') and then dividing by the count using '/'.

- **Built-in Data Structures**

Programming languages provide pre-defined ways to organize data (e.g., lists, dictionaries). These data structures come with standard operations, such as adding an element, removing an element, or searching for an element.

- **User-Defined Complex Data Structures**

  We can create sophisticated data organizations tailored to specific problems, such as trees, graphs, or custom classes. These are again built using more basic structures as building blocks.

1.3 What is a Program?

A program is fundamentally a sequence of precise instructions that a computer can execute to perform a specific task. Every program must address three key aspects:

1. **Sequence of Instructions**

   The program consists of simple but precise operations arranged in a logical order.

2. **Flow of Control**

   The program specifies which instruction should be executed next, including conditional branching and loops. This determines the order of execution and allows the program to react differently depending on data and conditions (e.g., "if the input is negative, print an error").

3. **Stopping Criteria**

   The program defines when to stop execution and return results. Without this, a program might run forever. For example, loops should eventually stop when a condition becomes false.

1.3.1 Program Characteristics

A good program should:

- Work correctly for all valid inputs, not just for a few examples.
- Terminate within reasonable time; an answer that never comes is not useful.
- Be understandable by humans, so that someone else (or you in the future) can read and modify it.
- Be maintainable and modifiable, allowing bug fixes and new features to be added without breaking everything else.

1.4 Getting Started with Python

Python is an interpreted language, meaning its code is executed line-by-line by an *interpreter* at runtime, rather than being compiled into machine code beforehand. To run Python programs, you can either:

- Use online interpreters (e.g., `https://www.programiz.com/python-programming/online-compiler/`)
- Install Python on your local machine and run programs from the terminal or a text editor.

Important: Avoid using IDEs or online platforms that autocomplete your code too at the beginning, as this can hinder learning. Typing code yourself helps in obtaining a deeper understanding.

1.4.1 First Program - Hello World

Let's start with the simplest program:

```
print("Hello World")
```

Output

```
Hello World
```

The `print` function displays text to the screen. It takes one or more values as input and shows them in the console. There are multiple ways to use `print`:

```
print("Hello")
print("World")
# Outputs: Hello on first line, World on second

print("Hello", "World")
# Outputs: Hello World (note the space between them)

print("Hello, World")
# Outputs: Hello, World (no automatic space)

# 'end' is used to control what print adds at the end.
# Here end=' ' adds a space instead of moving to a new line.

print("Hello", end=' ')    # Prints Hello and stays on same line
   ↪ with a space
print("world.")            # Prints next text on same line

# Output: Hello world.
```

Important Note: Quotation marks matter! `print("Hello")` prints the text Hello, while `print(hello)` would try to print the value of a variable named `hello`, which doesn't exist, causing an error.

Case sensitivity: Print is not the same as print. Python is case-sensitive.

1.5 Basic Data Types

Python supports several fundamental data types for storing simple values:

Type	Description / Example
`int`	Represents integers (whole numbers), e.g., `age = 22`
`float`	Represents decimal/real numbers, e.g., `weight = 50.2`
`bool`	Represents Boolean values `True/False`, e.g., `ispresent = True`
`str`	Represents text/strings, e.g., `name = "Alice"`

Example: Multiple data types together

```
temperature = 36          # int
pi_value = 3.14159        # float
is_logged_in = False      # bool
city = "Delhi"           # str

print(temperature, pi_value, is_logged_in, city)
```

```
Output: 36 3.14159 False Delhi
```

1.6 Variables and Memory

A variable is a named memory location that stores a value. When you declare a variable, Python reserves memory and creates a mapping between the variable name and that memory location.

```
age = 22            # int type
weight = 50.2       # float type
name = "Python"     # str type
is_student = True # bool type
```

Understanding memory is crucial: each variable points to a specific memory address where the value is stored. When `age = 22` and `weight = 50.2` are executed, Python allocates memory for the values 22 and `50.2`, and the names `age` and `weight` reference these locations. You can inspect this using the `id()` function:

```
x = 5
print(id(x))   # Prints memory address, e.g., 140734823585400
```

The exact memory address does not matter; it simply shows that Python is storing the value somewhere and the variable `x` refers to that location.

Example: Reference behavior

```
a = 100
b = a

print(id(a))
print(id(b))
```

Output:

```
140734239845456
140734239845456
```

Explanation:

- Two variables can point to same memory when values are same
- For example, a is the second largest if it is greater than one number and smaller than the other.

1.7 Operations on Data Types

1.7.1 Arithmetic Operations

Standard mathematical operations work on numeric types:

```
a = 5
b = 2
x = a + b       # Addition: 7
y = a - b       # Subtraction: 3
z = a * b       # Multiplication: 10
w = a / b       # Division: 2.5
q=  a // b      # Floor Division: 2
r = a % b       # Remainder (modulo): 1
p = a ** b      # Exponentiation: 25
```

Important: Use $*$ for multiplication, not $\times$ or $\cdot$.

Note: Division with / in Python 3 always produces a `float`, even when the result is an integer.

Rectangle example

```
length = 12
width = 5

area = length * width
perimeter = 2 * (length + width)
```

```
print("Area:", area)
print("Perimeter:", perimeter)
```

Output:

```
Area: 60
Perimeter: 34
```

1.7.2 String Operations

Strings support concatenation and repetition:

```
a = "Hello"
b = "World"
c = a + " " + b         # Concatenation: "Hello World"
d = a * 3               # Repetition: "HelloHelloHello"
```

The + operator on strings joins them, and * with an integer repeats the string that many times. The same symbol (like +) can mean different things depending on the data type: addition for integers vs concatenation for strings.

1.7.3 Boolean Operations

Logical operations combine Boolean values:

```
a = True
b = False
x = a and b     # Logical AND: False
y = a or b      # Logical OR: True
z = not b       # Logical NOT: True
```

Truth table for Boolean operations:

- `and` is `True` only if *all* operands are `True`.
- `or` is `False` only if *all* operands are `False`.
- `not` inverts the Boolean value.

Example:

```
has_id = True
has_ticket = True

can_enter = has_id and has_ticket
print(can_enter)
```

Output:

```
True
```

1.7.4 Comparison Operations

Comparisons produce Boolean results:

```
a = 5
b = 3
x = (a == b)    # Equal: False
y = (a != b)    # Not equal: True
z = (a > b)     # Greater than: True
w = (a < b)     # Less than: False
v = (a >= b)    # Greater or equal: True
u = (a <= b)    # Less or equal: False
```

These operations are often used in `if` statements and loop conditions (to be discussed later) to make decisions based on data.

Example: Comparison in decision making

```
marks = 78
cutoff = 75

print(marks > cutoff)
print(marks == cutoff)
```

Output:

```
True
False
```

1.8 Type Conversion

A data type specifies the kind of value a variable holds or the type of input being provided. Based on the data type, we can determine which operations and functions can be performed on that value.

```
temperature = 36
pi_value = 3.14159
is_logged_in = False
city = "Delhi"

print(type(temperature))     #Output: <class 'int'>
print(type(pi_value))        #Output: <class 'float'>
print(type(is_logged_in))    #Output: <class 'bool'>
type(type(city))             #Output: <class 'str'>
```

Python supports both implicit and explicit type conversions.

Implicit conversion happens automatically when it is safe, e.g., converting an `int` to `float` in an arithmetic expression:

```
a = 3
b = 4.5
c = a + b    # c becomes 7.5 (float)
```

Explicit conversion uses functions like `int`, `float`, or `str` when you decide you want to change the type:

```
x = "88"
y = int(x)          # y becomes 88 (int)

z = float(3.14)     # z becomes 3.14 (float)
w = str(42)         # w becomes "42" (str)

print(type(x))      # Output: <class 'str'>
print(type(y))      # Output: <class 'int'>
print(type(z))      # Output: <class 'float'>
```

To find a variable's type in Python, one can use the built-in `type()` function.

Important: Type conversion fails if it is not possible:

```
int("abc")      # ValueError: invalid literal
float("ten")    # ValueError: could not convert string to float
```

1.9 Input and Output

1.9.1 Output

Use the `print()` function to display information:

```
print("Hello, World!")
print("The answer is:", 42)
print("Multiple", "values", "here")
```

Each `print` call normally ends with a newline, so the next output starts on a new line. Commas automatically insert spaces between arguments.

1.9.2 Input

Use the `input()` function to read user input:

```
name = input("Enter your name: ")
print("Hello, " + name)

age_str = input("Enter your age: ")
age = int(age_str)  # Convert to integer
print("Next year you'll be:", age + 1)
```

Note: `input` always returns a `string`. For numeric operations, conversion is necessary. When you call `int(input("Enter your age: "))`, Python:

1. Calls `input`, which waits for user input and returns a string.
2. Passes that string to `int`, which converts it to an integer.
3. Assigns the integer to the variable.

Example: Input/Type conversion /Arithmetic/Output

```
name = input("Enter your name: ")
salary = float(input("Enter your monthly salary: "))

annual_salary = salary * 12

print("Hello", name)
print("Your annual salary is:", annual_salary)
```

Output:

```
Enter your name: Ravi
Enter your monthly salary: 50000
Hello Ravi
Your annual salary is: 600000.0
```

Chapter 2

Control Flow

2.1 Conditional Statements

Conditional statements allow programs to make decisions and execute different code based on conditions.

2.1.1 If-Else Structure

```
if condition:
    # Code executed if condition is True
    statement_1
    statement_2
elif other_condition:
    # Code executed if other_condition is True
    statement_3
else:
    # Code executed if all conditions are False
    statement_4
```

Note: Python uses *indentation* to define code blocks. Proper indentation is essential and not optional; all lines inside the same block must be aligned.

Example: Finding Maximum of Three Numbers

```
a = 6.6
b = 3.1
c = 2.4

if a >= b and a >= c:
    print("The largest number is", a)
elif b >= a and b >= c:
    print("The largest number is", b)
else:
    print("The largest number is", c)
```

Example: Finding the Second Largest of Three Distinct Numbers

Consider three distinct integers. The second largest number is the one that is greater than one number but smaller than the other.

```
a = 5.6
b = 5.0
c = 5.2

if ((a < b and a > c) or (a > b and a < c)):
    print(a, "is the second largest number")
elif ((b < a and b > c) or (b > a and b < c)):
    print(b, "is the second largest number")
else:
    print(c, "is the second largest number")
```

Explanation:

- A number is the second largest if it lies between the other two numbers.
- For example, a is the second largest if it is greater than one number and smaller than the other.
- Logical operators `and` and `or` are used to combine conditions.
- Since the numbers are distinct, exactly one condition will be true.

2.1.2 *While Loops*

A `While` loop is used to repeatedly execute a block of code as long as a given condition is satisfied (true).

The general form of a while loop is as follows:

```
while condition:
    # Code to repeat
    statement_1
    statement_2
    update_statement   # Must update condition
```

The token 'condition' in the above code is evaluated as a `Boolean`.

Flow:

1. The expression is evaluated (all operators, function calls, etc.).
2. The resulting value is converted using truth-value testing.
3. If it is `True`, the loop continues; otherwise, it stops.

Control Flow of a While Loop

When the program encounters `While` loops, it first evaluates the condition. If the condition is evaluated to `be true`, the statements inside the loop body are executed. After executing the loop body, the control returns to the test condition and the condition is evaluated again. This process continues until the test condition evaluates to `false`, at which point the loop terminates, and the control moves to the statement following the loop.

A `While` loop is an **entry-controlled loop**, which means that the test condition is evaluated before execution of the loop body. In case of a while loop, it is possible that the loop body may not get executed even once since we are checking condition before entering the loop. It is also possible that the loop may continue going on forever, in which case our program execution will not end.

Example:

```
while x - 5 :
    ...
```

Here:

- x - 5 is evaluated first.
- The result is then converted to `bool`.

Rules:

- 0, 0.0, `None`, `False`, empty containers → `False`
- Everything else → `True`.

Example: Print number from 1 to 4

```
i = 1
while i <= 4:
    print(i, end=" ")
    i = i + 1
```

Output:

```
1 2 3 4
```

Explanation

1. Initialize $i = 1$.
2. Start the loop by checking the condition $i \leq 4$:

- Print the current value of i.
- Increment i by 1.

3. When i becomes 5, the condition $i \leq 4$ is false and the loop terminates.

Example:

```
i = 10
while i < 5:
    print(i, end=" ")
    i = i + 1
print(100)
```

Output:

```
100
```

Explanation:

1. The variable i is initialized to 10.
2. The condition $i < 5$ is evaluated before the loop body executes.
3. Since the condition is false on the first check, the loop body is skipped.
4. Control immediately moves to the statement following the loop.

Example: Infinite Loop

```
i = 1
while i > 0:
    print(i, end=" ")
```

Output:

```
1 1 1 1 1 1 ...
```

Explanation:

1. The variable i is initialized to 1.
2. The condition $i > 0$ is evaluated before each iteration.
3. Since the value of i is always greater than 0 and is never modified inside the loop, the condition always remains true.
4. As a result, the loop body keeps executing repeatedly without termination.

Example: Output Even Numbers

```
n = int(input("Enter a number: "))
i = 2
while i <= n:
    print(i)
    i = i + 2
```

Important Considerations:

1. The loop condition must eventually become `False`, otherwise you get an infinite loop.
2. Always update the loop variable inside the loop.
3. Test loop termination logic carefully to avoid off-by-one errors.

Example: Square of the number

```
n = int(input("enter positive integer: "))
i = 0
answer = 0
while (i < n):
    answer = answer + n
    i = i + 1
print(answer)
```

Food for thought: What happens if the input is a negative number?

Example: Dynamically Typed in not always good.

```
n = input("Enter number : ")
i = 0
while (i < n):
    print("*"*n)
    i = i + 1
```

Output:

```
TypeError: '<' not supported between instances of 'int' and 'str'
```

Example: Nested loops

```
n = int(input("Enter first number : "))
m = int(input("Enter second number : "))
i = 0
while (i < n):
    j = 0
```

```
    while(j < m):
        print(i, j)
        j = j+1
    i = i+1
```

Output:

```
Enter first number: 2
Enter second number: 3
0 0
0 1
0 2
1 0
1 1
1 2
```

Visual Tool: `https://pythontutor.com/visualize.html#mode=edit`
You can visualize the execution flow of any python program with the help of this awesome tool.

Example: Greatest Common Divisor

```
a = int(input("first number : "))
b = int(input("second number : "))
g = 1
gcd = 1
while ((g <= a) and (gcd <= b)):
    # the gcd has to be a number between 1 and min(a,b)
    # we will search this space sequentially, starting with 1
    if ((a%g == 0) and (b%g == 0)):
        # if g divides both
        gcd = g
    g = g+1
print("GCD is",gcd)
```

Output:

```
enter number: 49
enter number: 21
GCD is 7
```

Explanation

The program reads two integers a and b and computes their greatest common divisor (GCD) by checking all possible divisors sequentially.

- The variable g is initialized to 1 and is used to iterate through all possible candidates for the GCD.

- For each g with $1 \leq g \leq \min(a, b)$, the program checks whether g divides both a and b.
- If g divides both numbers, the variable `gcd` is updated to g.
- After the loop finishes, `gcd` stores the largest number that divides both a and b, which is printed.

This algorithm is correct but inefficient, since it tests all integers from 1 to $\min(a, b)$, giving a time complexity of $O(\min(a, b))$. Efficiency is crucial in algorithm design, especially for large inputs, and we will later study a much more efficient method (the Euclidean algorithm) for computing the GCD.

Try this program with large (nine digit) numbers.

Exercise: Greatest Common Divisor

What are the outputs of the three programs below?

Program A

```
a = int(input("first number : "))
b = int(input("second number : "))
g = 1
gcd = 1
while ((g <= a) and (gcd <= b)):
    if ((a%g == 0) and (b%g == 0)):
        gcd = g
    g = g+1
print("GCD is",gcd)
```

Program B

```
a = int(input("first number : "))
b = int(input("second number : "))
g = 1
gcd = 1
while ((g <= a) and (gcd <= b)):
    if ((a%g == 0) and (b%g == 0)):
        gcd = g
    g = g+1
    print("GCD is",gcd)
```

Program C

```
a = int(input("first number : "))
b = int(input("second number : "))
g = 1
gcd = 1
while ((g <= a) and (gcd <= b)):
    if ((a%g == 0) and (b%g == 0)):
```

```
            gcd = g
            print("GCD is",gcd)
        g = g+1
```

Testing: Greatest Common Divisor

Given a program that takes as input three positive integers a,b,g, and outputs "Yes" if g is the GCD of a and b.

```
a = int(input("Enter a : "))
b = int(input("Enter b : "))
g = int(input("Enter g : "))

i = g+1

check = True

# check g divides both a and b
if(a%g != 0):
    check = False
if(b%g != 0):
    check = False

# check g is the largest number that divides both a and b
while((i <= a) and (i <= b)):
    if((a%i == 0) and (b%i == 0)):
        check = False
    i = i+1

if(check):
    print("Yes")
else:
    print("No")
```

Think: Do we need to continue this loop when `check = False`?
Thought: No, we can just `break` out of the loop at that point.

Some Common Mistakes while using while-loop

- Forgetting to update the loop control variable, which may result in an infinite loop.
- Write an incorrect condition that never becomes false.
- Using assignment (=) instead of comparison (==) in the condition.

When to Use a While Loop?

A while loop is preferred when:

- The number of iterations is not known in advance.
- Loop termination depends on a dynamic condition (example, you take some input from user inside the loop and that variable appears in the loop condition)
- The input is processed until a specific condition is met.

2.2 Breaking Out of Loops

The `break` statement exits the **innermost** loop immediately.

```
password = "secret"
attempts = 0
while attempts < 5:
    user_input = input("Enter password: ")
    if user_input == password:
        print("Correct!")
        break
    attempts = attempts + 1
```

Here, the loop stops when the correct password is entered or when the number of attempts reaches 5. `break` is useful when you do not know in advance exactly when you will find what you are looking for.

Re-Testing: Greatest Common Divisor

We can now use this primitive to `break` out of the loop early.

```
a = int(input("Enter a : "))
b = int(input("Enter b : "))
g = int(input("Enter g : "))

i = g+1

check = True

# check g divides both a and b
if(a%g != 0):
    check = False
if(b%g != 0):
    check = False

# check g is the largest number that divides both a and b
while((i <= a) and (i <= b)):
    if((a%i == 0) and (b%i == 0)):
        check = False
        break
    i = i+1
```

```
if(check):
    print("Yes")
else:
    print("No")
```

Example: Primality testing

```
n = int(input("enter number: "))
i = 2
prime = True
while(i<n):
    if (n%i == 0):
        prime=False
        break
    i=i+1

if(prime):
    print(n,"is prime")
else:
    print(n,"is not prime")
```

Output:

```
enter number: 15
No
```

Output:

```
enter number: 31
Yes
```

Homework: Can you make this more efficient?

Exercise: Take a break

```
n = 5
i = 1
while (i<=n):
    print("Output 1")
    if(i*i > n):
        print("Output 2")
        break
        print("Output 3")
    print("Output 4")
    i = i+1
print("Output 5")
```

What is the output of this program?

2.3 For Loops

A for loop is used to repeatedly execute a block of code for each item in a sequence

2.3.1 Range Function

The `range` function generates sequences of integers:

- `range(n)` generates `0, 1, 2, ..., n-1`.
- `range(m, n)` generates `m, m+1, ..., n-1`.
- `range(m, n, s)` generates numbers starting at `m`, incrementing by `s`, up to but not including `n`.

Note: range (m, n, s) can be used to decrement when your s is negative.

2.3.2 For Loop Structure

```
for variable in range(n):
    # Code to repeat
    statement1
    statement2
```

2.3.3 Examples

```
for i in range(10):
    print(i)          # Prints numbers 0 to 9

for i in range(5, 10):
    print(i)          # Prints numbers 5 to 9

for i in range(0, 10, 2):
    print(i)          # Prints 0, 2, 4, 6, 8

for i in range(10,0,-2):
    print(i)          # Prints 10, 8, 6, 4, 2
```

2.3.4 Example: Computing Factorial

```
n = int(input("Enter a number: "))
factorial = 1

for i in range(1, n + 1):
    factorial = factorial * i
```

```
print("Factorial of", n, "is", factorial)
```

Each iteration multiplies the current `factorial` by the current integer `i`, building up the result step by step.

2.3.5 Break in a `for` Loop

The `break` statement works the same way in both `while` and `for` loops. It immediately terminates the *innermost* loop in which it appears.

```
mysum = 0
for i in range(5, 11, 2):
    mysum = mysum + i
    if (i == 9):
        break
        mysum = mysum + 1
print(mysum)  # mysum = 5 + 7 + 9 = 21
```

Explanation:

- The loop starts with $i = 5$ and increments by 2 each time.
- Thus, the values of i are: $5, 7, 9$.
- At each iteration, the current value of i is added to `mysum`.
- When $i = 9$, the condition `(i == 9)` evaluates to `True`.
- The `break` statement is executed, which immediately exits the loop.

Important Observation:

- The statement `mysum = mysum + 1` is placed *after* the `break`.
- Since `break` exits the loop instantly, this line is never executed.
- Therefore, the final value of `mysum` is:

$$5 + 7 + 9 = 21.$$

2.3.6 Primality Testing

```
n = int(input("enter number: "))
prime = True
for i in range(2, n):
    if (n % i == 0):
        prime = False
```

```
        break

if (prime):
    print(n, "is prime")
else:
    print(n, "is not prime")
```

Explanation:

- We initially assume that the number is prime by setting `prime = True`.
- The loop checks all integers i such that $2 \leq i < n$.
- If any i divides n (i.e., $n \bmod i = 0$), then:
  - `prime` is set to `False`.
  - The `break` statement terminates the loop immediately.
- If no divisor is found, the value of `prime` remains `True`.

Important Note (Case $n = 2$):

- When $n = 2$, the loop becomes `range(2, 2)`.
- This produces an empty sequence since there is no integer that starts at 2 and is strictly less than 2.
- Hence, the loop body is never executed.
- The value of `prime` remains equal to its initial value `True`.
- Therefore, the program correctly identifies 2 as a prime number.

Remark:

- A similar situation occurs with ranges such as `range(5, 1)` when the step size is positive.
- Such ranges are valid in Python but generate empty sequences.
- Consequently, the loop body is skipped without producing any error.

Food for thought: Can a composite number have all of its prime divisors to be greater than $\sqrt{n}$? Based on this try to improve the primality testing.

Homework: Try executing all the examples given in while loop, using for loop.

2.4 *for vs while: When to Use What?*

Both `for` and `while` loops are used for repetition, but they are suited for different situations. Choosing the appropriate loop improves clarity, readability, and correctness of code.

When to Use a `for` Loop

A `for` loop is preferred when:

- The number of iterations is known in advance.
- You are iterating over a sequence.
- You want automatic control of the loop variable.
- You do not want to manually update a counter.

```
# Iterating over a range
for i in range(5):
    print(i)
```

When to Use a `while` Loop

A `while` loop is preferred when:

- The number of iterations is not known beforehand.
- The loop depends on a condition that changes dynamically.
- The loop continues until a particular event occurs.
- You need finer control over the stopping condition.

```
# Running until user enters 0
n = int(input("Enter number: "))
while n != 0:
    print(n)
    n = int(input("Enter number: "))
```

2.5 Example: Nested Loops

Nested loops mean a loop inside another loop.

```
# Understanding the control flow
for i in range(3):
    for j in range(3):
        print("i =", i, "j =", j)
        # this is a comment
```

Step-by-Step Execution:

- The outer loop runs as `range(3)`, so i takes the values:

$$i = 0, 1, 2.$$

- For each fixed value of i, the inner loop runs completely.
- The inner loop also runs as `range(3)`, so j takes the values:

$$j = 0, 1, 2.$$

- Thus, for every single value of i, the inner loop executes three times.

Execution Pattern:

- When $i = 0$:

$$(0,0), (0,1), (0,2)$$

- When $i = 1$:

$$(1,0), (1,1), (1,2)$$

- When $i = 2$:

$$(2,0), (2,1), (2,2)$$

Total Number of Executions:

- The outer loop runs 3 times.
- The inner loop runs 3 times for each outer iteration.
- Therefore, the `print` statement executes:

$$3 \times 3 = 9 \text{ times.}$$

Below we present an example of nested loops to generate patterns.

```
# Print a right-angled triangle of stars
n = int(input("Enter height: "))

for i in range(1, n + 1):          # i controls the row number
    for j in range(1, i + 1):      # j controls number of stars in
        ↪ a row
        print("*", end="")         # end="" avoids newline after
            ↪ each star
    print()                        # moves to next line
```

For example, if `n = 4`, the output is:

```
*
**
***
****
```

The outer loop decides how many rows to print. For each row `i`, the inner loop prints exactly `i` stars on that row.

2.5.1 Example: Prime numbers less than n

```
n = int(input("Enter the upper bound: "))
for k in range(2, inp) :
    flag = True
    for i in range(2, k) :
        if (k%i == 0):
            flag = False
            break
    if (flag):
        print(k, "is prime")
```

Output:

```
Enter the upper bound: 13
2 is prime
3 is prime
5 is prime
7 is prime
11 is prime
```

This is a nested loop in which we are iterating the primality testing.

Homework: What would happen if the `flag=True` statement is written on top of the bigger for loop?

Chapter 3

Data Structures: Strings

Strings are sequences of characters. They are **immutable**, meaning once created, they cannot be **modified in place**.

Example

```
s = "Hello World"
print(s)
s1 = s + '!'
print(s1)
s2 = s + "!" * 2
print(s)
```

Output:

```
Hello World
Hello World!
Hello World!!
```

Explanation:

1. You can assign a Python variable with a string using single quotes, double quotes, and even triple quotes.
2. Two strings could be added using + symbol
3. Multiplication of a string by a number give a string repeated that many times.

Let us see a few more operations.

1. s1 == s2: True if both strings are equal, this operation is case sensitive.
2. we can print the string using print(s)
3. when we take user input using s = input("Enter a number") it return a sting, you can convert it to integer using int(s). (Be cautious as if user typed a character it will raise error)
4. similarly float(s) will convert string to float. (Be cautious again)

3.1 String Indexing

The ith character of string s can be accessed using s[i]

```
s = "Hello World"
print("The Third character is", s[2])

print(s[11])
```

Output:

```
The third character is l
IndexError: string index out of range.
```

Explanation:

1. s[2] is 3rd character of string ”Hello World” that is ’l’. Python use 0 based indexing.
2. Index should be **less** than length of string other wise will raise IndexError.

3.2 String Reverse Indexing

The ith character of string s **from the end** can be accessed using s[-i].

```
s = "Hello World"
print("The last character is", s[-1])
```

Output:

```
The last character is d
```

Explanation:

1. The first character from the end is d.

3.3 Length of string

The length of string s can be accessed using len(s).

```
s = "Hello World"
print(len(s))
```

Output:

```
11
```

Explanation:

1. ”Hello World” is made up of 11 characters.

3.4 String Slicing

We can get a substring of a string using slicing as s[start: end], where the substring starts at position **start** till **end**-1. Note, we do not go till the **end**, but one character less. We can also drop **end** after: that will give the substring till the end of our original string. Let's see the examples.

```
s = "Hello World"
print("s[2:5] is", s[2:5])
print ("s[2:] is", s[2:])
```

Output:

```
s[2:5] is llo
s[2:] is llo World
```

Explanation:

1. We get characters at index 2, 3 and 4 of Hello World.
2. we print till the end, starting from the index 3.
3. Note that we use 0 based indexing.

3.5 Example: Extract Year from dd/mm/yyyy

Write a program that takes as input a date in `dd/mm/yyyy` format, and outputs the year.

```
s = input("enter date in dd/mm/yyyy format: ")
t = s[-4:]       # t = s[6:] or t = s[6:10] also works
print(t)
```

Explanation:

- The input is stored as a string.
- The year occupies the last 4 characters.
- Using slicing `s[-4:]`, we extract the last four characters.

3.6 Example: Count Number of Vowels

Write a program that takes as input a five-letter word and outputs the number of vowels in the word.

```
s = input("enter a five-letter word: ")

count = 0
```

```
if ((s[0] == 'a') or (s[0] == 'e') or (s[0] == 'i') or (s[0] == '
   ↪ o') or (s[0] == 'u')):
    count = count + 1

if ((s[1] == 'a') or (s[1] == 'e') or (s[1] == 'i') or (s[1] == '
   ↪ o') or (s[1] == 'u')):
    count = count + 1

if ((s[2] == 'a') or (s[2] == 'e') or (s[2] == 'i') or (s[2] == '
   ↪ o') or (s[2] == 'u')):
    count = count + 1

if ((s[3] == 'a') or (s[3] == 'e') or (s[3] == 'i') or (s[3] == '
   ↪ o') or (s[3] == 'u')):
    count = count + 1

if ((s[4] == 'a') or (s[4] == 'e') or (s[4] == 'i') or (s[4] == '
   ↪ o') or (s[4] == 'u')):
    count = count + 1

print("The number of vowels is", count)
```

Important Note: String equality is case-sensitive. If some characters are uppercase, the answer may be incorrect.

3.7 Example: Using `lower()` and `upper()`

- `s.lower()` converts all alphabets to lowercase.
- `s.upper()` converts all alphabets to uppercase.

```
t = input("enter a five-letter word: ")
s = t.lower()

count = 0

if ((s[0] == 'a') or (s[0] == 'e') or (s[0] == 'i') or (s[0] == '
   ↪ o') or (s[0] == 'u')):
    count = count + 1

if ((s[1] == 'a') or (s[1] == 'e') or (s[1] == 'i') or (s[1] == '
   ↪ o') or (s[1] == 'u')):
    count = count + 1

if ((s[2] == 'a') or (s[2] == 'e') or (s[2] == 'i') or (s[2] == '
   ↪ o') or (s[2] == 'u')):
```

```
    count = count + 1

if ((s[3] == 'a') or (s[3] == 'e') or (s[3] == 'i') or (s[3] == '
   ↪ o') or (s[3] == 'u')):
    count = count + 1

if ((s[4] == 'a') or (s[4] == 'e') or (s[4] == 'i') or (s[4] == '
   ↪ o') or (s[4] == 'u')):
    count = count + 1

print("The number of vowels is", count)
```

Observation: We are repeating the same code five times. There should be a better way to do this using loops.

3.8 Exercise: Output of the Code

```
string = "abcde"

for i in range(len(string) + 1):
    print(string[i])
```

Output:

```
a
b
c
d
e
ERROR!
Traceback (most recent call last):
IndexError: string index out of range
```

Explanation:

- `len("abcde")` is 5.
- `range(len(string) + 1)` produces values 0 to 5.
- Valid indices are 0 to 4.
- When `i = 5`, Python raises an `IndexError`.

3.9 For loop with strings.

For Loops Can Iterate Over Characters of a String

```
ss = "abcde"

for i in ss:
    print(i)
```

Output:

```
a
b
c
d
e
```

Explanation: The loop directly iterates over each character of the string without using indices.

3.10 Example: Program to Count the Number of Spaces in a String

```
s = input("enter string: ")

count = 0

for i in s:
    if (i == ' '):
        count = count + 1

print("number of spaces is", count)
```

3.11 Example: Program to Detect Repeated Characters in a String

```
flag = False

s = input("enter string: ")

for i in range(len(s)):
    for j in range(i + 1, len(s)):
        if (s[i] == s[j]):
```

```
            flag = True

if flag:
    print("The string has repeated characters.")
else:
    print("All characters are unique.")
```

Explanation:

- We compare each character with the remaining characters.
- If any match is found, we set `flag = True`.
- After checking all pairs, we print the result.

Chapter 4

Functions

4.1 Introduction to Functions

A function is a reusable block of code that performs a specific task. Functions allow us to:

- **Organize code** into logical, manageable pieces
- **Avoid repetition** of similar code
- **Enhance readability** by using meaningful function names
- **Enable abstraction** by hiding implementation details

Conceptually, a function is like a small machine: you give it input, it does some work, and it returns an output.

4.2 Defining and Calling Functions

4.2.1 Function Definition

```
def function_name(parameter1, parameter2):
    """
    This is a docstring explaining what the function does.
    It takes parameter1 and parameter2 as input.
    It returns the result.
    """
    # Function body
    result = parameter1 + parameter2
    return result
```

4.2.2 Function Call

```
# Calling the function with actual arguments
output = function_name(5, 3)
print(output)  # Prints 8
```

- **Formal Parameters**: The variables listed in the function definition.
- **Actual Arguments**: The values passed when calling the function.
- **Return Statement**: Specifies what value the function sends back to the caller.

Example: Computing Maximum/Minimum

```
def max_of_two(x, y):
    """Return the maximum of two numbers."""
    if x >= y:
        return x
    else:
        return y

result = max_of_two(5, 10)
print(result)  # Prints 10
```

Formal parameter names (x, y) can be different from names of variables used when calling the function; Python matches them by position.

Example: Computing absolute value

```
def my_abs(n):
    if n < 0:
        return -n  # If negative, make it positive
    else:
        return n   # If positive or zero, return as is

# Example usage
val = -15
result = my_abs(val)
print(result) # Output: 15
```

4.2.3 Example: Functions Calling Functions

```
def square(num):
    return num * num

def power_four(num):
    # Calls the square function on the result of square(num)
    temp = square(num)
    return square(temp)

# Example usage
print(power_four(2)) # Output: 16 (2*2 = 4, then 4*4 = 16)
```

Functions call others to solve sub-tasks, reducing repetition and organizing complex logic into smaller, reusable code blocks.

4.2.4 Loops inside Functions

```
def count_divisors(n):
    count = 0
    # Iterate from 1 up to n (inclusive)
    for i in range(1, n + 1):
        # Check if 'i' divides 'n' evenly
        if n % i == 0:
            count = count + 1
    return count

# Example usage
print(count_divisors(10)) # Output: 4 (Divisors are 1, 2, 5, 10)
```

Example: String Processing

```
def starts_with(full_str, prefix):
    # A prefix cannot be longer than the string itself
    if len(prefix) > len(full_str):
        return False

    # Check only the first len(prefix) characters
    for i in range(len(prefix)):
        if full_str[i] != prefix[i]:
            return False # Mismatch found

    return True # Loop finished with no mismatches

# Example usage
s1 = "Python Programming"
s2 = "Pyth"
print(starts_with(s1, s2)) # Output: True
```

4.3 Variable Scope

Core Idea: Variable scope tells you which parts of the code can access a variable.

In Python, variables have:

1. **Local Scope**: Variables defined inside a function, accessible only within that function.
2. **Enclosing Scope**: Variables in an outer function (for nested functions).
3. **Global Scope**: Variables defined at the top level of the program, accessible everywhere.

Example: Local Scope *vs* Global Scope

```
global_var = 10  # Global scope

def my_function():
    local_var = 5  # Local scope
    print(global_var)  # Can access global variable
    print(local_var)   # Can access local variable
    return global_var + local_var

my_function()
# print(local_var)  # ERROR - local_var not accessible
```

When Python exits a function, it "forgets" the values assigned to local variables. Below is another example:

```
def f(x):
    y = 1
    x = x + y
    print("In function: x =", x, ", y =", y)
    return x

x = 3
y = 2
z = f(x)
print("Outside: x =", x, ", y =", y, ", z =", z)

# Output:
# In function: x = 4, y = 1
# Outside: x = 3, y = 2, z = 4
```

The key insight is that changes to local variables do not affect variables with the same name outside the function. In the example, the `x` inside function `f` is a separate local copy. Modifying it does not change the outer `x`. Understanding this protects you from subtle bugs when variable names are reused.

4.3.1 How Python keeps track of variables (and why scope exists)

You can think of Python as keeping a table of names that maps *variable names* to their *current values*. This is how Python knows what `x`, `minval`, or `maxval` mean at any point in the program.

Now, here is where scope comes in.

1. **Each function call gets its own local scope**
   When a function is called, Python creates a new, separate name table just for that function call. This table represents the local scope of the function.

   - It starts with the values passed as arguments (the actual arguments are assigned to the function's parameters)
   - It can also see names from the outer (global) scope for reading
   - But any variable you assign inside the function goes into this local table

   Example:

```
x = 10  # global

def f(a):
    b = a + 1    # local variable
    print(x)     # reads global x
    return b

f(5)
```

   Here:

   - `a` and `b` live in the local scope of the function.
   - `x` lives in the global scope

2. **Local scope exists only while the function runs**
   While the function is executing, its local variables exist. As soon as the function reaches return (or finishes), Python throws away that local name table. That means:

   - Local variables do not exist outside the function.
   - Each function call gets a fresh, independent local scope.

   Example:

```
def g():
    y = 42
    return y

g()
# print(y)  #  Error: y does not exist here
```

The variable y was created in g's local scope, and that scope disappeared after g() returned.

3. **A function can read variables from the outer (global) scope**
   Example:

```
x = 100

def h():
    print(x)  # uses global x

h()
```

But if you assign to a name inside the function, Python treats it as local unless you say otherwise with global.

4.3.2 *Variable Scoping Examples*

1. **Program**:

```
def f(x):
    sq = x*x
    x= 3 sq= 9
    print("In function")
    print("x=", x, "sq=", sq)
    return sq

x=3
z = f(x)
print("Outside function")
print("x=", x, "sq=", sq, "z=", z)
```

Output:

```
In function
x= 3 sq= 9
Outside function
ERROR!

NameError: name 'sq' is not
defined
```

When Python exits the function, it "deletes" the local variables.

2. **Program**:

```
def f(x):
    sq = x*x
    x= 3 sq= 9
    print("In function")
    print("x=", x, "sq=", sq)
```

```
    return sq

x=3
sq = 0
z = f(x)
print("Outside function")
print("x=", x, "sq=", sq, "z=", z)
```

Output:

```
In function
x= 3 sq= 9
Outside function
x= 3 sq= 0 z= 9
```

Inside a function, Python has a local copy of variable.

3. **Program**:

```
x = "global_x"
def outer():
    x = "enclosing_x"
    def inner_1():
        x = "local_x"
        print(x)
    def inner_2():
        print(x)
    inner_1()
    inner_2()
outer()
print(x)
```

Output:

```
local_x
enclosing_x
global_x
```

4. **Program**:

```
def circle_area(rad):
    ans = pi*(rad**2)
    return ans

def sphere_volume(rad):
    ans = 4*pi*(rad**3)/3
    return ans

pi = 3.142
x = 10
```

```
print(circle_area(x))
print(sphere_volume(x))
```

Output:

```
314.2
4189.333333333333
```

Here pi is a global variable (accessible to both functions). So, there is no error.

5. **Program**:

```
def circle_area(rad):
    ans = pi*(rad**2)
    return ans

def sphere_volume(rad):
    ans = 4*pi*(rad**3)/3
    return ans

pi = 3.142
print(circle_area(x))
x = 10
print(sphere_volume(x))
```

Output:

```
Line 2, in circle_area
NameError: name 'pi' is not defined
```

Here pi is a global variable. But, it is defined after the function circle_area.

6. **Program**:

```
def F(x):
    def G():
        x = "abc"
        print("in G, x =", x)
    print("in F, x =", x)
    x = x*10
    G()
    print("in F, x =", x)
    return x

x=3
z = F(x)
print("outside, x =", x)
print("outside, z =", z)
```

Output:

```
in F, x = 3
in G, x = abc
in F, x = 30
outside, x = 3
outside, z = 30
```

Homework

Compare how variable scoping rules will manifest in the following two programs:
Program 1:

```
def addOne(x):
    y=x+1
    print(y)

x=3
y=3

addOne(x)
```

Program 2:

```
def addOne(x):
    y=y+1
    print(y)

x=3
y=3

addOne(x)
```

Final Remarks on Variable Scoping: To avoid scoping issues, make sure that your functions do not use variables outside the scope.

Chapter 5

Recursions

Recursion is a programming technique where a function calls itself to solve a problem by breaking it into smaller instances of the same problem. This is based on the **divide-and-conquer methodology**:

1. **Divide**: Break the problem into smaller instances of the same problem.
2. **Conquer**: Solve the smaller instances recursively.
3. **Combine**: Merge solutions to solve the original problem.

A recursive solution requires:

1. **Base Case**: The condition under which the recursion stops.
2. **Recursive Case**: How the problem is reduced to a smaller version of itself.

5.1 Computing the sum of squares

Let us start with a simple example: we want a function that, for any positive integer n, returns

$$\mathsf{sum\_squares}(n) = \sum_{i=1}^{n} i^2.$$

Of course, we can compute this iteratively using a while/for loop, but for pedagogical purposes, let us do this using recursion. Before coding this up in Python, we need to identify the recursive structure in this problem. Here, the recursive structure can be identified easily.

$$\mathsf{sum\_squares}(n) = \begin{cases} 1 \text{ if } n == 1 \\ n^2 + \mathsf{sum\_sqares}(n-1) \text{ otherwise} \end{cases}$$

Once we have this recursive structure, we can code it up in Python as follows.

```
def sum_squares(n):
    if(n==1):
        return 1
    else:
        return (n*n + sum_squares(n-1))
```

5.2 Factorial computation using recursion

Mathematically, factorial of an integer n can be represented as:

$$n! = \begin{cases} 1 & \text{if } n = 1 \\ n \times (n-1)! & \text{otherwise} \end{cases}$$

Thus, the following function computes factorial of any given positive integer recursively.

```
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n - 1)

# Example
print(factorial(5))  # Prints 120
```

5.3 Computing Fibonacci Sequence

The Fibonacci sequence is defined by the recursive relation:

$$f(n) = \begin{cases} 1 & \text{if } n = 1, 2 \\ f(n-1) + f(n-2) & \text{otherwise} \end{cases}$$

```
def fib(n):
    if n ==1 or n==2:
        return 1
    else:
        return fib(n - 1) + fib(n - 2)

# Example
print(fib(4))   # Prints 3
print(fib(5))   # Prints 5
```

Note: This naive recursive implementation is inefficient for large n because it recalculates the same values many times. For example, computing `fib(5)` requires 15 function calls.

5.4 Square Root using Recursive Binary Search

Consider the problem of computing for a given positive integer n, the value $\lfloor \sqrt{n} \rfloor$ recursively (note that this is the floor of the square root and not the square root itself).

An naive approach to this problem can be seen as follows:

```
n = int(input())
i = 0
while ((i+1)**2 <= n):
    i = (i+1)

print(i)
```

Notice that we are actually evaluating the square of all number from 1 to $\lfloor n^{1/2} \rfloor$. Can we avoid evaluating some numbers? This will allow us to reach the solution in fewer number of steps.

Recursion helps here. The key idea to solve this problem recursively is to maintain an interval $[left, right]$ such that

$$left \leqslant \lfloor \sqrt{n} \rfloor < right.$$

and keep reducing the length of the interval, until we reach an interval range which is satisfied only by one integer. Initially, we know that the answer must lie between 1 and n, so we start with the interval $[1, n+1]$.

Define a function `sqrt_floor(n, left, right)` with the precondition that $\lfloor \sqrt{n} \rfloor \in [left, right - 1]$.

- The base case is when the interval has collapsed to size two, i.e., when $left + 1 = right$; then $left$ must be the floor of $\sqrt{n}$.
- Otherwise, we compute
$$mid = \left\lfloor \frac{left + right}{2} \right\rfloor$$
and compare mid^2 with n.
  - If $n < mid^2$, the answer lies in the left subinterval $[left, mid - 1]$, and we recurse there;
  - If $mid^2 \leqslant n$, the answer lies in $[mid, right - 1]$, and we recurse in the right sub-interval.

```
def sqrt_floor(n, left=1, right=None):
    if right is None:
        right = n+1     # invariant: answer in [left, right-1]

    # Base case: range collapsed
    if left + 1 == right:
        return left

    mid = (left + right) // 2
    if (n < mid * mid):
        return sqrt_floor(n, left, mid)
    else:
        return sqrt_floor(n, mid, right)

print(sqrt_floor(15))   # 3
print(sqrt_floor(16))   # 4
print(sqrt_floor(101))  # 10
```

Try to draw the ranges on the Integer Line and observe how left and right approach closer to eachother.

Attempt this: Try to fill in the blanks to complete the code that lets you get the floor of the kth root of an integer n:

```
n = int(input())

def kthRoot(n, k, L, R):
    if L+1 == R:
        return ________
    mid = (L+R)//2
    if (n < mid**k):
        return ______________________________
    else:
        return ______________________________

ans = kthRoot(n,2,0,n+1) + kthRoot(n,5,0,n+1)
print(ans)
```

Food for Thought: Square root is a monotonic function, can such a recursive solution be applied, say, to find out $\lfloor n^{1/2} \rfloor - \lfloor n^{1/3} \rfloor$?

5.5 GCD Computation

Below is a naive approach to compute the greatest common divisor (GCD) of two integers.

```
def gcd_naive(a, b):
    """Find GCD by checking all numbers from 1 to min(a,b)."""
    g = 1
```

```
    for i in range(1, min(a, b) + 1):
        if a % i == 0 and b % i == 0:
            g = i
    return g
```

Euclid's Method is an efficient method for computing GCD. Key idea of this approach is the following claim.

Claim. *For any two integers a, b $(a > b)$ with r $(= a \% b)$ as remainder, we have*

$$\gcd(a, b) = \gcd(r, b).$$

```
def gcd(a, b):
    """Find GCD using Euclid's algorithm."""
    if (a < b):
        a, b = b, a
    r = a % b
    if (r == 0): return b
    else:  return gcd(b,r)

# Example
print(gcd(140,89))     # prints 1
print(gcd(1071, 462))  # prints 21
```

Euclid's algorithm terminates much faster than the naive approach, especially for large numbers.

Explanation of Euclid's GCD Algorithm

1. Why the Algorithm Terminates At each recursive step, the pair (a, b) is replaced by (b, r), where:

$$r = a \bmod b$$

By definition of the modulo operation:

$$0 \leq r < b$$

Thus, the second argument strictly decreases in every recursive call. Since b is a non-negative integer and decreases each time, it must eventually become 0.

When $r = 0$, the recursion stops and the algorithm returns b.

Therefore, the algorithm always terminates.

2. Why the Algorithm is Correct The correctness is based on the fundamental property:

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

Reason:
If a number d divides both a and b, then it must also divide:

$$a - qb$$

for any integer q.
Since:

$$a - qb = a \bmod b$$

any common divisor of a and b is also a divisor of b and $a \bmod b$, and vice versa.
Thus, the set of common divisors remains unchanged at each step.
When $r = 0$, we have:

$$a = qb$$

so b divides a, meaning b is the greatest common divisor.
Hence, the algorithm correctly returns $\gcd(a, b)$.

3. Why Euclid's Algorithm is Much More Efficient **Naive Method:**
The naive method checks every number from 1 to $\min(a, b)$ to see if it divides both numbers.
For example, if:

$$a = 1{,}000{,}000 \quad \text{and} \quad b = 500{,}000$$

the naive algorithm may check up to 500,000 numbers.
This means it can require a very large number of operations.
Time complexity:

$$O(\min(a, b))$$

Euclid's Algorithm:
Euclid's algorithm does something much smarter.
Instead of checking many possible divisors, it repeatedly replaces:

$$(a, b) \to (b, a \bmod b)$$

The important observation is:

$$a \bmod b < b$$

So the second number strictly becomes smaller at every step.

Intuitive Idea:
Each step significantly reduces the size of the numbers.
Example:

$$\gcd(1071, 462)$$

$$1071 \bmod 462 = 147$$

$$462 \bmod 147 = 21$$

$$147 \bmod 21 = 0$$

We reached the answer in only **3 steps**.

Even for very large numbers (millions or billions), Euclid's algorithm typically finishes in only a small number of steps.

Time Complexity:
It can be proven that the number of steps is proportional to:

$$O(\log(\min(a, b)))$$

This means:

- If the numbers grow 10 times larger,
- the number of steps increases only by a small amount.

Why This Is Much Faster

- Naive method: checks potentially hundreds of thousands (or millions) of values.
- Euclid's method: reduces the numbers quickly and needs only a few recursive calls.

Therefore, Euclid's algorithm is exponentially faster than the naive method and is the standard way to compute GCD.